

Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский технологический
университет «МИСиС»

Институт ИТКН
Кафедра АСУ

**ОТЧЕТ ПО УЧЕБНОЙ/ ПРОИЗВОДСТВЕННОЙ
ПРАКТИКЕ**

Выполнил:
ст. гр. БИВТ-24-9
Кукишев Артём Вадимович
Подпись:

Москва 2026

Содержание

- Введение
- 1 Постановка задачи
- 2 Практическая часть
 - Архитектура решения
 - Проектирование базы данных
 - Программный интерфейс (API)
 - Модель авторизации
 - Ролевая модель
 - Транзакции и защита от коллизий
 - Пользовательский интерфейс
 - Тестирование
- Заключение
- Список использованных источников

Введение

В современной разработке программного обеспечения задачи командной работы и управления процессами всё чаще решаются с помощью систем визуального планирования. Одним из наиболее распространённых подходов является методология Kanban, при которой работа представлена в виде карточек, перемещающихся по колонкам, отражающим стадии выполнения. Такие системы, как Trello и Kaiten, широко применяются в командах для организации совместной деятельности.

Целью данной практики являлась разработка собственного многопользовательского веб-сервиса Kanban-досок, функционально аналогичного существующим решениям. Основной акцент был сделан на проектировании реляционной модели данных, построении программного интерфейса с механизмом аутентификации и разграничения прав доступа, а также на обеспечении целостности данных при одновременной работе нескольких пользователей.

Разработанный сервис позволяет пользователям регистрироваться, создавать доски, приглашать участников с различными ролями, управлять колонками и карточками, перемещать задачи между стадиями методом перетаскивания, назначать исполнителей и сроки, а также обсуждать задачи в комментариях. Все действия фиксируются в журнале изменений.

1 Постановка задачи

В рамках практики необходимо реализовать веб-сервис управления задачами на основе Kanban-досок. Требования к обязательному минимуму (MVP) сформулированы следующим образом:

- 1) Пользователи и доступ: регистрация и вход в систему; доска имеет владельца и участников.
- 2) Доски: создание, редактирование, удаление; получение списка досок пользователя.
- 3) Колонки: создание, редактирование, удаление.
- 4) Карточки: создание и редактирование заголовка и описания; перемещение между колонками методом drag-and-drop; порядок карточек в колонке; назначение исполнителя; установка срока выполнения.
- 5) Комментарии: добавление комментариев к карточкам.
- 6) Роли: не менее трёх ролей с разграничением прав по принципу минимальных привилегий.

Помимо функциональных требований, предъявляются требования к безопасности и надёжности: пароли пользователей должны храниться в виде хэшей, операции над данными — выполняться в транзакциях, а система должна быть защищена от конфликтов при одновременном редактировании доски несколькими клиентами. Выбор языка программирования, базы данных и технологий пользовательского интерфейса остаётся на усмотрение разработчика.

2 Практическая часть

Архитектура решения

Сервис построен по классической трёхзвенной архитектуре, включающей клиентское приложение, сервер приложений и базу данных. Клиентское приложение реализовано в виде одностраничного приложения (SPA) на библиотеке React. Серверная часть построена на фреймворке FastAPI и предоставляет программный интерфейс в стиле REST. В качестве системы управления базами данных используется PostgreSQL. Общая схема взаимодействия компонентов приведена на Рис. 1.

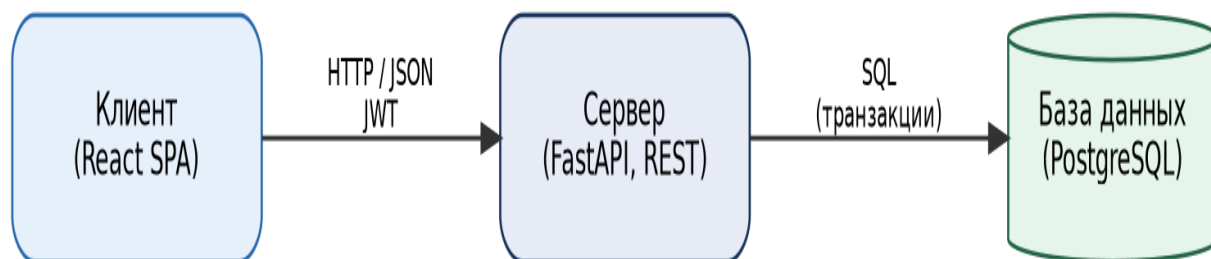


Рис. 1. Верхнеуровневая архитектура сервиса

Ключевой принцип архитектуры состоит в том, что клиент не обращается к базе данных напрямую — все операции проходят через серверную часть. Это обеспечивает единую точку контроля доступа, валидации входных данных и управления транзакциями. Взаимодействие клиента и сервера осуществляется по протоколу HTTP в формате JSON, аутентификация — с помощью токенов JWT.

Проектирование базы данных

Модель данных включает семь основных сущностей: пользователи, доски, участники досок, колонки, карточки, комментарии и журнал изменений. Связь «пользователь — доска» имеет тип «многие-ко-многим» и разрешена через промежуточную таблицу участников, в которой дополнительно хранится роль пользователя на конкретной доске. Отдельная

таблица журнала изменений фиксирует историю действий над доской. Диаграмма «сущность — связь» представлена на Рис. 2.

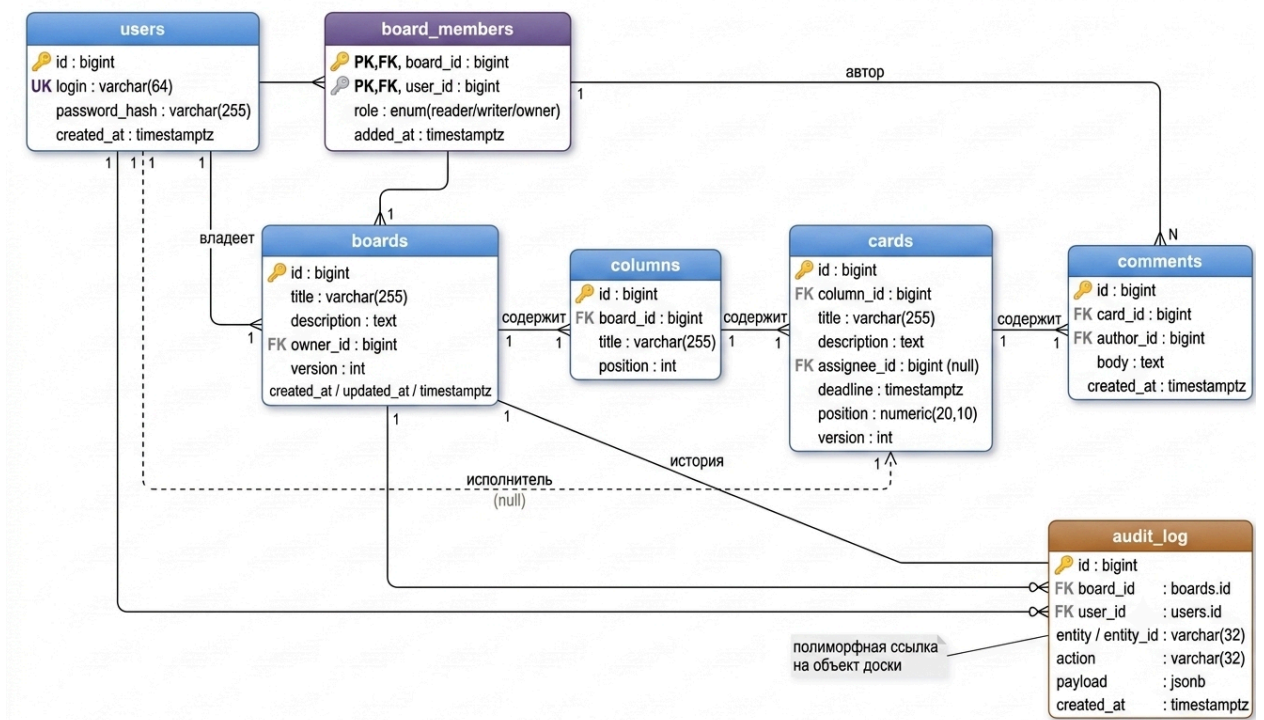


Рис. 2. ER-диаграмма базы данных

Первичные ключи всех таблиц — суррогатные, целочисленные. Внешние ключи снабжены правилами каскадного удаления: при удалении доски удаляются её колонки, карточки и комментарии, при удалении пользователя — снимается его назначение исполнителем. Для ускорения частых выборок созданы индексы: по идентификатору пользователя в таблице участников (список досок пользователя), по паре «доска — позиция» для колонок и по паре «колонка — позиция» для карточек (отрисовка доски в правильном порядке), а также по полям исполнителя, срока и времени создания комментариев.

Позиция карточки внутри колонки хранится как число с плавающей точкой. Такой подход, известный как дробное индексирование (fractional indexing), позволяет вставлять карточку между двумя соседними, вычисляя её позицию как среднее арифметическое позиций соседей. Благодаря этому

перемещение карточки изменяет ровно одну строку и не требует пересчёта позиций всех остальных карточек колонки.

При проектировании запросов применялись соединения таблиц (JOIN), группировка (GROUP BY), оконные функции и подзапросы. Например, список досок пользователя с числом карточек формируется соединением таблиц досок, участников, колонок и карточек с последующей группировкой по доске.

Программный интерфейс (API)

Программный интерфейс реализован в стиле REST. Все ответы и входные данные описываются схемами и автоматически валидируются, на основе чего формируется интерактивная документация в формате OpenAPI. Все конечные точки, за исключением регистрации и входа, требуют передачи токена в заголовке запроса. Перечень основных методов приведён в Таблице

Таблица 1 — Основные методы программного интерфейса

Метод	Конечная точка	Назначение	Мин. роль
POST	/api/auth/register	Регистрация	—
POST	/api/auth/login	Вход, выдача токена	—
GET / POST	/api/boards	Список и создание досок	участник
GET / PATCH / DELETE	/api/boards/{id}	Операции с доской	reader / writer / owner
GET / POST	/api/boards/{id}/members	Управление участниками	reader / owner
GET / POST	/api/boards/{id}/columns	Колонки	reader / writer
GET / POST	/api/boards/{id}/cards	Карточки	reader / writer
POST	/api/cards/{id}/move	Перемещение карточки	writer
GET / POST	/api/cards/{id}/comments	Комментарии	reader / writer
GET	/api/boards/{id}/audit	Журнал изменений	owner

Модель авторизации

Аутентификация пользователей основана на токенах JWT (JSON Web Token). При входе пользователь передаёт логин и пароль, а в ответ получает подписанный токен, который затем прикладывается к каждому защищённому запросу. Сервер проверяет подпись и срок действия токена, извлекает идентификатор пользователя и определяет его права. Жизненный цикл токена показан на Рис. 3.

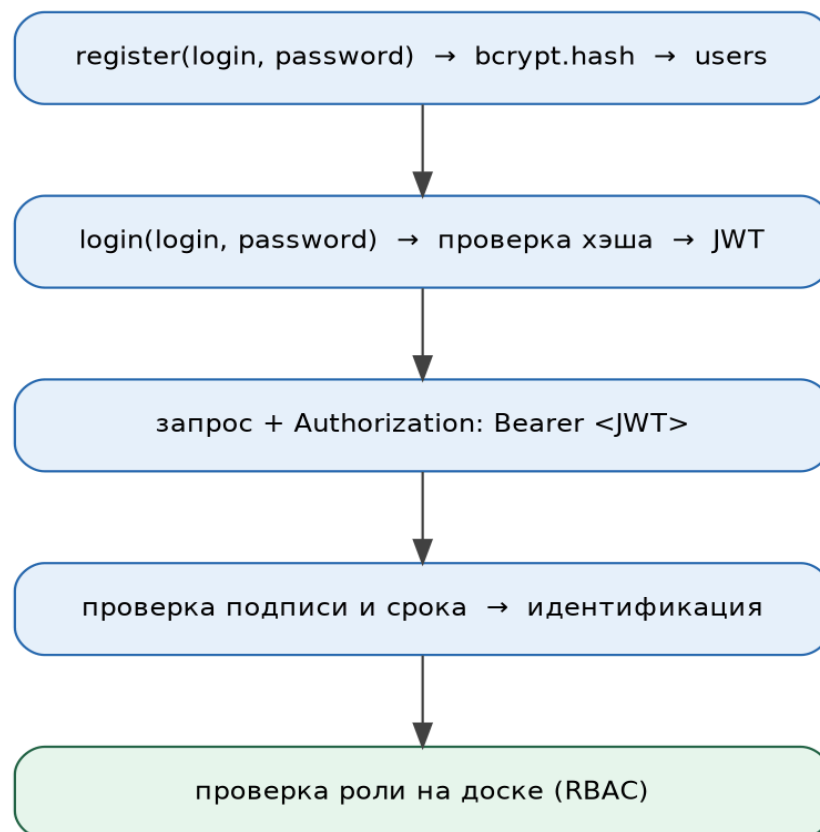


Рис. 3. Жизненный цикл токена авторизации

Пароли пользователей никогда не хранятся в открытом виде. При регистрации пароль хэшируется алгоритмом `bcrypt` со встроенной солью, и в базе данных сохраняется только полученный хэш. При входе введённый пароль повторно хэшируется и сравнивается с сохранённым значением. Такой подход исключает компрометацию паролей даже в случае получения злоумышленником доступа к базе данных.

Ролевая модель

Для разграничения прав доступа выбрана модель управления доступом на основе ролей (RBAC). Пользователю на уровне доски назначается одна из трёх ролей, образующих иерархию по объёму прав. Реализован принцип минимальных привилегий: доска, к которой у пользователя нет доступа, возвращает ответ «не найдено», что не раскрывает сам факт её существования. Права ролей приведены в Таблице 2.

Таблица 2 — Роли и права доступа

Роль	Чтение	Редактирование	Администрирование	Управление ролями
Читатель (reader)	Да	Нет	Нет	Нет
Писатель (writer)	Да	Да	Нет	Нет
Владелец (owner)	Да	Да	Да	Да

Выбор в пользу RBAC, а не более сложной модели на основе атрибутов (ABAC), обусловлен характером предметной области. Полный набор прав в системе описывается тремя фиксированными ролями, привязанными к паре «пользователь — доска»; динамических атрибутивных условий, при которых был бы оправдан переход к ABAC, в задаче нет. Таким образом, RBAC обеспечивает необходимую выразительность при существенно меньшей сложности реализации и сопровождения.

Транзакции и защита от коллизий

Операции, затрагивающие несколько строк базы данных, выполняются в рамках одной транзакции. Характерным примером является перемещение карточки: изменение колонки, обновление позиции, увеличение версии и запись в журнал изменений фиксируются атомарно. Для защиты от конфликтов при одновременном редактировании применяется механизм оптимистичной блокировки на основе поля версии. Клиент передаёт известную ему версию объекта; если она не совпадает с текущей версией в базе данных, сервер возвращает ответ «конфликт» (код 409), после чего

клиент перечитывает актуальное состояние и повторяет действие. Последовательность взаимодействия показана на Рис. 4.

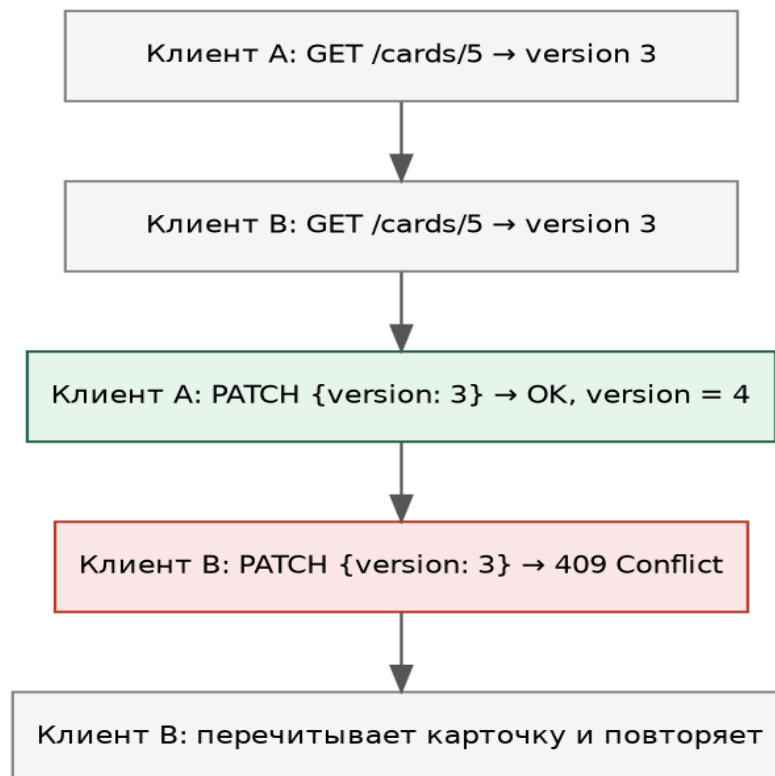


Рис. 4. Защита от коллизий (оптимистичная блокировка)

Данный подход, в отличие от пессимистичной блокировки, не удерживает записи базы данных заблокированными на время работы клиента и лучше масштабируется при большом числе пользователей, что оправдано при относительно редких конфликтах на доске.

Пользовательский интерфейс

Клиентская часть реализована в виде одностраничного приложения на библиотеке React. Интерфейс включает страницу входа и регистрации, список досок пользователя и рабочую область доски с колонками и карточками. Перемещение карточек между колонками и внутри колонки реализовано методом перетаскивания (drag-and-drop) с оптимистичным обновлением интерфейса: карточка визуально перемещается немедленно, а при получении ответа о конфликте состояние доски перечитывается с сервера. Детали

карточки — описание, исполнитель, срок и комментарии — открываются в модальном окне.

Тестирование

Основные сценарии работы сервиса проверены через программный интерфейс. Результаты проверки приведены в Таблице 3; все проверки пройдены успешно.

Таблица 3 — Результаты проверки основных сценариев

Сценарий	Ожидаемый результат	Итог
Вход по логину и паролю	Выдача токена	Пройдено
Создание и перемещение карточки	Смена колонки, рост версии	Пройдено
Перемещение с устаревшей версией	Конфликт (409)	Пройдено
Запись читателем	Отказ в доступе (403)	Пройдено
Доступ к чужой доске	Не найдено (404)	Пройдено
Запрос без токена	Требуется вход (401)	Пройдено
Фиксация действий в журнале	Записи созданы	Пройдено

Заключение

В ходе практики спроектирован и реализован полнофункциональный веб-сервис Kanban-досок. Выполнены все требования обязательного минимума: реализованы регистрация и авторизация пользователей, управление досками, колонками и карточками, перемещение карточек методом перетаскивания, назначение исполнителей и сроков, комментирование задач и ролевое разграничение доступа.

Отработаны проектирование реляционной модели данных с корректными связями и индексами, построение программного интерфейса с аутентификацией на основе токенов JWT и ролевой моделью RBAC, обеспечение транзакционной целостности и защиты от конкурентных изменений с помощью оптимистичной блокировки, а также сборка многосервисного приложения в единую воспроизводимую конфигурацию.

В качестве направлений дальнейшего развития можно выделить добавление тегов и меток карточек, чек-листов и вложений, полнотекстового поиска по карточкам, а также шаблонов досок. За время прохождения практики были применены теоретические знания при решении практической задачи разработки программного обеспечения.

Список использованных источников

- 1) Документация фреймворка FastAPI, <https://fastapi.tiangolo.com/>
- 2) Документация СУБД PostgreSQL, <https://www.postgresql.org/docs/>
- 3) Документация библиотеки SQLAlchemy, <https://docs.sqlalchemy.org/>
- 4) Introduction to JSON Web Tokens, <https://www.jwt.io/introduction>
- 5) Документация библиотеки React, <https://react.dev/learn>
- 6) PostgreSQL: Transaction tutorial,
<https://www.postgresql.org/docs/current/tutorial-transactions.html>